

**Московский государственный технический
университет им. Н. Э. Баумана**

Факультет «Информатика и системы управления»
Кафедра ИУ5 «Системы обработки информации и управления»

Курс «Парадигмы и конструкции языков программирования»
Отчет по рубежному контролю №2

Выполнила:
Студент группы ИУ5-31Б
Сербенюк Полина
Проверил:
Гапанюк Ю. Е.

2025 г.

РК2 реализован в той же директории, что и РК1. В связи с этим директория переименована rk1-2. Листинг кода для лр.2 представлен со страницы [7].

ЛР № 1.

Classes.py

```
class Conductor:
```

```
    """Дирижер"""
```

```
    def __init__(self, id, name, experience, orchestra_id):
```

```
        self.id = id
```

```
        self.name = name
```

```
        self.experience = experience # стаж работы в годах
```

```
(количественный признак)
```

```
        self.orchestra_id = orchestra_id
```

```
class Orchestra:
```

```
    """Оркестр"""
```

```
    def __init__(self, id, name):
```

```
        self.id = id
```

```
        self.name = name
```

```
class ConductorOrchestra:
```

```
    """
```

```
    'Дирижеры оркестра' для реализации
```

```
связи многие-ко-многим
```

```
"""
```

```
    def __init__(self, orchestra_id, conductor_id):
```

```
        self.orchestra_id = orchestra_id
```

```
        self.conductor_id = conductor_id
```

data.py

```
from classes import Conductor, Orchestra, ConductorOrchestra
```

```
orchestras = [
```

```
    Orchestra(1, 'Большой симфонический оркестр'),
```

```

Orchestra(2, 'Камерный оркестр "Виртуозы"'),
Orchestra(3, 'Симфонический оркестр филармонии'),
Orchestra(4, 'Студийный оркестр'),
Orchestra(5, 'Эстрадный оркестр'),
]

# Дирижеры
conductors = [
    Conductor(1, 'Светланов', 40, 1),
    Conductor(2, 'Плетнев', 35, 2),
    Conductor(3, 'Федосеев', 45, 3),
    Conductor(4, 'Гергиев', 38, 3),
    Conductor(5, 'Баршай', 30, 4),
    Conductor(6, 'Китаенко', 42, 1),
    Conductor(7, 'Синайский', 36, 5),
]

conductors_orchestras = [
    ConductorOrchestra(1, 1),
    ConductorOrchestra(1, 6),
    ConductorOrchestra(2, 2),
    ConductorOrchestra(3, 3),
    ConductorOrchestra(3, 4),
    ConductorOrchestra(4, 5),
    ConductorOrchestra(5, 7),

    # Дополнительные связи для многие-ко-многим
    ConductorOrchestra(1, 3), # Светланов также дирижирует
    Симфоническим оркестром
    ConductorOrchestra(2, 7), # Плетнев также дирижирует
    Эстрадным оркестром
    ConductorOrchestra(3, 1), # Федосеев также дирижирует
    Большим симфоническим
    ConductorOrchestra(4, 2), # Баршай также дирижирует
    Камерным оркестром
    ConductorOrchestra(5, 5), # Синайский также дирижирует

```

Студийным оркестром
]

requests.py

```
from operator import itemgetter
import data
```

```
def create_one_to_many():
    """Создание соединения один-ко-многим"""
    return [(c.name, c.experience, o.name)
            for o in data.orchestras
            for c in data.conductors
            if c.orchestra_id == o.id]
```

```
def create_many_to_many():
    """Создание соединения многие-ко-многим"""
    many_to_many_temp = [(o.name, co.orchestra_id,
                           co.conductor_id)
                          for o in data.orchestras
                          for co in data.conductors_orchestras
                          if o.id == co.orchestra_id]
```

```
    return [(c.name, c.experience, orch_name)
            for orch_name, orch_id, cond_id in many_to_many_temp
            for c in data.conductors if c.id == cond_id]
```

```
def request_1(one_to_many):
```

```
    print('Задание A1')
    print('Список всех дирижеров и оркестров, отсортированный
по оркестрам:')
    res_11 = sorted(one_to_many, key=itemgetter(2)) # сортировка
по названию оркестра
    for item in res_11:
```

```

    print(f' {item[0]} - {item[1]} лет ({item[2]})')
print()
return res_11

def request_2(one_to_many):
    """
    Задание A2

    print('Задание A2')
    print('Список оркестров с суммарным стажем дирижеров,
отсортированный по суммарному стажу:')

    # Используем словарь для группировки по оркестрам
    orchestra_experience = {}
    for conductor_name, experience, orchestra_name in
one_to_many:
        if orchestra_name not in orchestra_experience:
            orchestra_experience[orchestra_name] = 0
            orchestra_experience[orchestra_name] += experience

    # Сортировка по суммарному стажу (по убыванию)
    res_12 = sorted(orchestra_experience.items(), key=itemgetter(1),
reverse=True)

    for orchestra_name, total_experience in res_12:
        print(f' {orchestra_name}: {total_experience} лет')
    print()
    return res_12

def request_3(many_to_many):

    print('Задание A3')
    print(
        'Список всех оркестров, у которых в названии присутствует

```

слово "оркестр", и список работающих в них дирижеров:'))

```
# Используем словарь для группировки дирижеров по оркестрам
res_13 = {}

# Фильтруем оркестры, содержащие "оркестр" в названии
filtered_orchestras = [o for o in data.orchestras if 'оркестр' in o.name.lower()]

for orchestra in filtered_orchestras:
    # Находим всех дирижеров для данного оркестра
    orchestra_conductors = [conductor_name
                             for conductor_name, experience, orch_name in
many_to_many
                             if orch_name == orchestra.name]
    # Убираем дубликаты
    unique_conductors = list(set(orchestra_conductors))
    if unique_conductors:
        res_13[orchestra.name] = unique_conductors

for orchestra_name, conductors_list in res_13.items():
    print(f' {orchestra_name}:')
    for conductor in conductors_list:
        print(f' - {conductor}')
print()
return res_13
```

```
def main():
    """Основная функция для выполнения всех запросов"""
    # Создаем соединения
    one_to_many = create_one_to_many()
    many_to_many = create_many_to_many()

    # Выполняем запросы
```

```
request_1(one_to_many)
request_2(one_to_many)
request_3(many_to_many)
```

```
if __name__ == '__main__':
    main()
```

ЛР № 2.

main.py

```
import requests
```

```
if __name__ == '__main__':
```

```
    requests.main()
```

test-main.py

```
import sys
```

```
import os
```

```
# Добавляем родительскую директорию в путь для импортов
sys.path.insert(0, os.path.abspath(os.path.join(os.path.dirname(__file__), '..')))
```

```
import requests
```

```
import data
```

```
class TestRequests:
```

```
    """Тесты для запросов из requests.py"""
```

```
    def setup_method(self):
```

```
        """Настройка перед каждым тестом"""
```

```
        self.one_to_many = requests.create_one_to_many()
```

```
        self.many_to_many = requests.create_many_to_many()
```

```
    def test_create_one_to_many(self):
```

```
        """Тест создания связи один-ко-многим"""
```

```
        result = requests.create_one_to_many()
```

```

# Проверяем что результат не пустой
assert len(result) > 0

# Проверяем структуру данных
for item in result:
    assert len(item) == 3 # (имя дирижера, стаж, имя оркестра)
    assert isinstance(item[0], str) # имя дирижера
    assert isinstance(item[1], int) # стаж
    assert isinstance(item[2], str) # имя оркестра

# Проверяем что все дирижеры привязаны к оркестрам
conductors_in_result = [item[0] for item in result]
assert 'Светланов' in conductors_in_result
assert 'Плетнев' in conductors_in_result

def test_request_1_sorted_by_orchestra(self):
    """Тест запроса 1: сортировка по оркестрам"""
    result = requests.request_1(self.one_to_many)

    # Проверяем что результат отсортирован по названию оркестра
    orchestra_names = [item[2] for item in result]

    # Проверяем что оркестры идут в алфавитном порядке
    assert orchestra_names == sorted(orchestra_names)

    # Проверяем конкретные данные
    first_item = result[0]
    assert isinstance(first_item[0], str) # имя дирижера
    assert isinstance(first_item[1], int) # стаж
    assert isinstance(first_item[2], str) # имя оркестра

def test_request_2_sum_experience(self):
    """Тест запроса 2: суммарный стаж по оркестрам"""
    result = requests.request_2(self.one_to_many)

    # Проверяем структуру результата
    assert len(result) > 0

    # Проверяем что это список кортежей (имя оркестра, суммарный стаж)
    for orchestra_name, total_exp in result:
        assert isinstance(orchestra_name, str)
        assert isinstance(total_exp, int)
        assert total_exp > 0 # стаж должен быть положительным

```



```

# Проверяем что результат отсортирован по убыванию стажа
experiences = [exp for _, exp in result]
assert experiences == sorted(experiences, reverse=True)

# Проверяем корректность подсчета
# Для оркестра с id=1 (Большой симфонический) есть 2 дирижера:
Светланов(40) и Китаенко(42)
orchestra_exp_dict = dict(result)
assert orchestra_exp_dict.get('Большой симфонический оркестр') == 82 #
40 + 42

def test_request_3_filter_orchestra_keyword(self):
    """Тест запроса 3: фильтрация по слову 'оркестр'"""
    result = requests.request_3(self.many_to_many)

    # Проверяем что это словарь
    assert isinstance(result, dict)

    # Проверяем что ключи содержат слово "оркестр"
    for orchestra_name in result.keys():
        assert 'оркестр' in orchestra_name.lower()

    # Проверяем что для каждого оркестра есть список дирижеров
    for orchestra_name, conductors in result.items():
        assert isinstance(conductors, list)
        assert len(conductors) > 0
        for conductor in conductors:
            assert isinstance(conductor, str)

    # Проверяем конкретный оркестр
    assert 'Большой симфонический оркестр' in result
    assert 'Светланов' in result['Большой симфонический оркестр']

def test_request_1_with_empty_data(self):
    """Тест запроса 1 с пустыми данными"""
    empty_one_to_many = []
    result = requests.request_1(empty_one_to_many)
    assert result == []

def test_request_2_with_single_conductor(self):
    """Тест запроса 2 с одним дирижером"""
    test_data = [('Дирижер1', 10, 'Оркестр1')]
    result = requests.request_2(test_data)

```

```

assert len(result) == 1
assert result[0][0] == 'Оркестр1'
assert result[0][1] == 10

def test_main_function_runs_without_errors(self):
    """Тест что основная функция выполняется без ошибок"""
    try:
        requests.main()
        # Если дошли сюда без исключений - тест пройден
        assert True
    except Exception as e:
        assert False, f"main() вызвала исключение: {e}"

def test_data_integrity():
    """Тест целостности данных"""
    # Проверяем что данные загружаются корректно
    assert len(data.orchestras) == 5
    assert len(data.conductors) == 7
    assert len(data.conductors_orchestras) == 12

    # Проверяем типы данных
    for conductor in data.conductors:
        assert hasattr(conductor, 'id')
        assert hasattr(conductor, 'name')
        assert hasattr(conductor, 'experience')
        assert hasattr(conductor, 'orchestra_id')
        assert isinstance(conductor.experience, int)

    for orchestra in data.orchestras:
        assert hasattr(orchestra, 'id')
        assert hasattr(orchestra, 'name')

if __name__ == "__main__":
    print("Запуск тестов...")

    # Создаем экземпляр тестового класса
    test_instance = TestRequests()

    # Запускаем setup
    test_instance.setup_method()

    # Запускаем тесты по одному

```

```

tests = [
    ("test_create_one_to_many", test_instance.test_create_one_to_many),
    ("test_request_1_sorted_by_orchestra",
test_instance.test_request_1_sorted_by_orchestra),
    ("test_request_2_sum_experience",
test_instance.test_request_2_sum_experience),
    ("test_request_3_filter_orchestra_keyword",
test_instance.test_request_3_filter_orchestra_keyword),
    ("test_data_integrity", test_data_integrity),
    ("test_main_function_runs_without_errors",
test_instance.test_main_function_runs_without_errors),
]

```

```

passed = 0
failed = 0

```

```

for test_name, test_func in tests:

```

```

    try:

```

```

        if test_name.startswith("test_"):

```

```

            # Это метод класса

```

```

            test_func()

```

```

        else:

```

```

            # Это обычная функция

```

```

            test_func()

```

```

        print(f"✅ {test_name}: PASSED")

```

```

        passed += 1

```

```

    except AssertionError as e:

```

```

        print(f"❌ {test_name}: FAILED - {e}")

```

```

        failed += 1

```

```

    except Exception as e:

```

```

        print(f"❌ {test_name}: ERROR - {type(e).__name__}: {e}")

```

```

        failed += 1

```

```

print(f"\nИтого: {passed} пройдено, {failed} упало")

```

```

if failed == 0:

```

```

    print("🎉 Все тесты прошли успешно!")

```

```

    sys.exit(0)

```

```

else:

```

```

    print("💣 Некоторые тесты не прошли")

```

```

    sys.exit(1)

```

вывод на экран:

```
Запуск тестов...
✅test_create_one_to_many: PASSED
Задание A1
Список всех дирижеров и оркестров, отсортированный по оркестрам:
    Светланов - 40 лет (Большой симфонический оркестр)
    Китаенко - 42 лет (Большой симфонический оркестр)
    Плетнев - 35 лет (Камерный оркестр "Виртуозы")
    Федосеев - 45 лет (Симфонический оркестр филармонии)
    Гергиев - 38 лет (Симфонический оркестр филармонии)
    Баршай - 30 лет (Студийный оркестр)
    Синайский - 36 лет (Эстрадный оркестр)

✅test_request_1_sorted_by_orchestra: PASSED
Задание A2
Список оркестров с суммарным стажем дирижеров, отсортированный по суммарному стажу:
    Симфонический оркестр филармонии: 83 лет
    Большой симфонический оркестр: 82 лет
    Эстрадный оркестр: 36 лет
    Камерный оркестр "Виртуозы": 35 лет
    Студийный оркестр: 30 лет
✅test_request_2_sum_experience: PASSED
Задание A3
Список всех оркестров, у которых в названии присутствует слово "оркестр", и список работающих в них дирижеров:
    Большой симфонический оркестр:
        - Светланов
        - Федосеев
        - Китаенко
    Камерный оркестр "Виртуозы":
        - Плетнев
        - Синайский
    Симфонический оркестр филармонии:
        - Светланов
        - Гергиев
        - Федосеев
    Студийный оркестр:
        - Плетнев
        - Баршай
    Эстрадный оркестр:
        - Синайский
        - Баршай
```

✅ test_request_3_filter_orchestra_keyword: PASSED

✅ test_data_integrity: PASSED

Задание A1

Список всех дирижеров и оркестров, отсортированный по оркестрам:

Светланов - 40 лет (Большой симфонический оркестр)
Китаенко - 42 лет (Большой симфонический оркестр)
Плетнев - 35 лет (Камерный оркестр "Виртуозы")
Федосеев - 45 лет (Симфонический оркестр филармонии)
Гергиев - 38 лет (Симфонический оркестр филармонии)
Баршай - 30 лет (Студийный оркестр)
Синайский - 36 лет (Эстрадный оркестр)

Задание A2

Список оркестров с суммарным стажем дирижеров, отсортированный по суммарному стажу:

Симфонический оркестр филармонии: 83 лет
Большой симфонический оркестр: 82 лет
Эстрадный оркестр:
- Синайский
- Баршай

✅ test_main_function_runs_without_errors: PASSED

Итого: 6 пройдено, 0 упало

👌 Все тесты прошли успешно!